

http://10.23.130.212/



WiFi : POCO X3 GT

PASSWORD : 12345678

PROJECT_LUMA_PKW · SYSTEM WALKTHROUGH

LUMA

ไล่ Flow และโค้ดของทั้ง 4 โหนด ตั้งแต่ผู้ใช้กดสร้างภาพจนได้ภาพกลับมา

14 ตุลาคม 2569 · 14 Oct 2026

โค้ดอ้างอิง: frontend-app 87ddda1 · backend 1cc127a · ai-node 6a6217c · devops cd6c9ce

NODE 1
DevOps · Nginx Gateway
bankjunjan · 10.23.130.212:80

NODE 2
Frontend · HTML / JS /CSS
wxnxtxr · 10.23.130.6:5500

NODE 3
Backend · FastAPI + DB
Pongkarm · 10.23.130.9:8000

NODE 4
AI · Stable Diffusion Forge
Kong19565 · 10.23.130.112:7860

ผู้ใช้ทำอะไรกับ LUMA ได้บ้าง

txt2img

สร้างภาพจากข้อความ

เลือกโมเดล, LoRA, sampler, steps, CFG, seed, ขนาด 256–768 px

POST /generations

img2img

แก้ภาพจากภาพต้นฉบับ

อัปโหลดภาพก่อน แล้วปรับ denoise ว่าจะให้ต่างจากต้นฉบับแค่ไหน

POST /uploads → /generations

inpaint

ระบายแก้เฉพาะจุด

วาด mask บน canvas แล้วส่ง mask ขาวดำไปพร้อมภาพ

mask.png → /uploads → /ai/edit

studio

เครื่องมือแต่งภาพ 4 อย่าง

จับท่าทาง (pose), ภาพลายเส้น (sketch), เน้นสีเดียว (splash), ตัดพื้นหลัง (remove-bg)

POST /api/tools/{tool}

history

ประวัติงาน

ดูงานย้อนหลัง, ใช้ค่าเดิมสร้างใหม่, ลบงาน, หยุดงานที่กำลังรัน

GET /generations · DELETE · /cancel

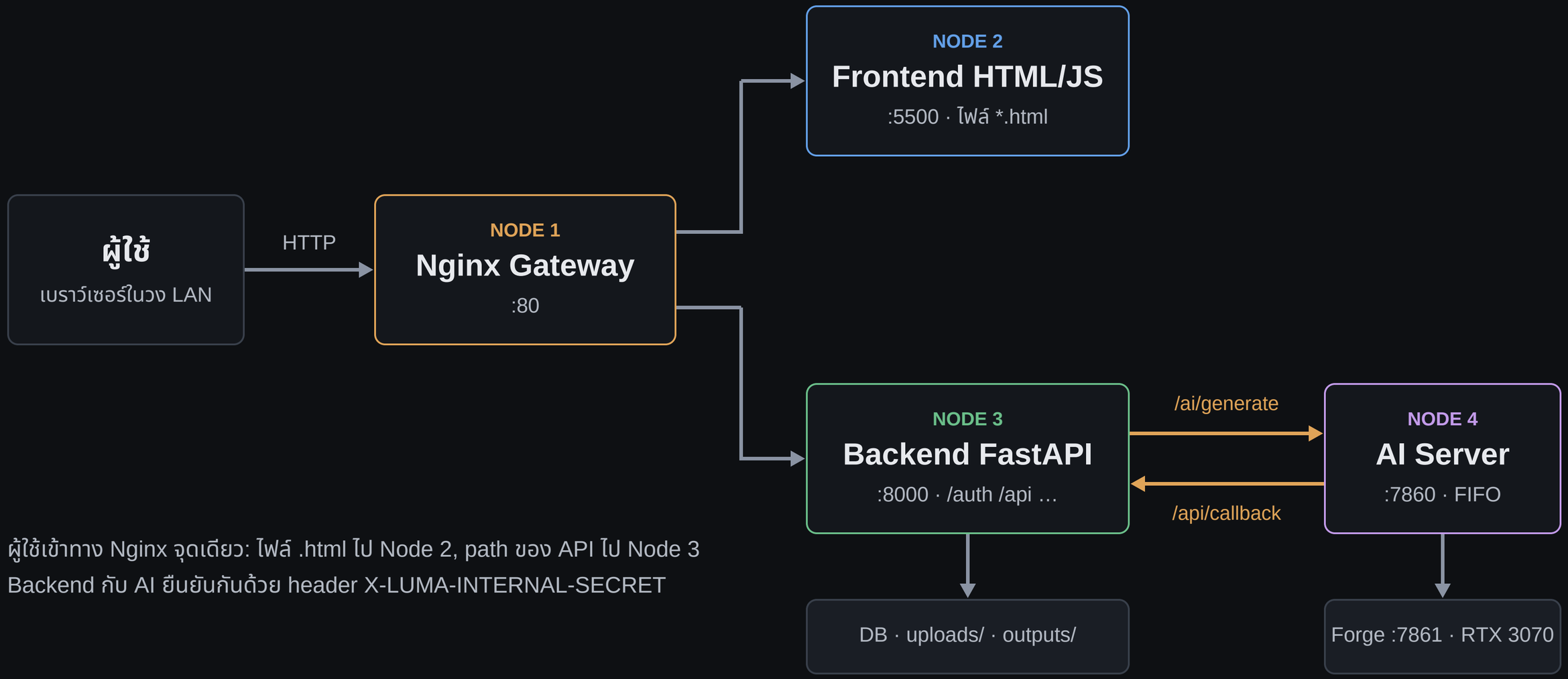
admin

แผงแอดมิน

สถิติ, ผู้ใช้, ระงับบัญชี, audit log, สิทธิ์ reviewer / admin / owner

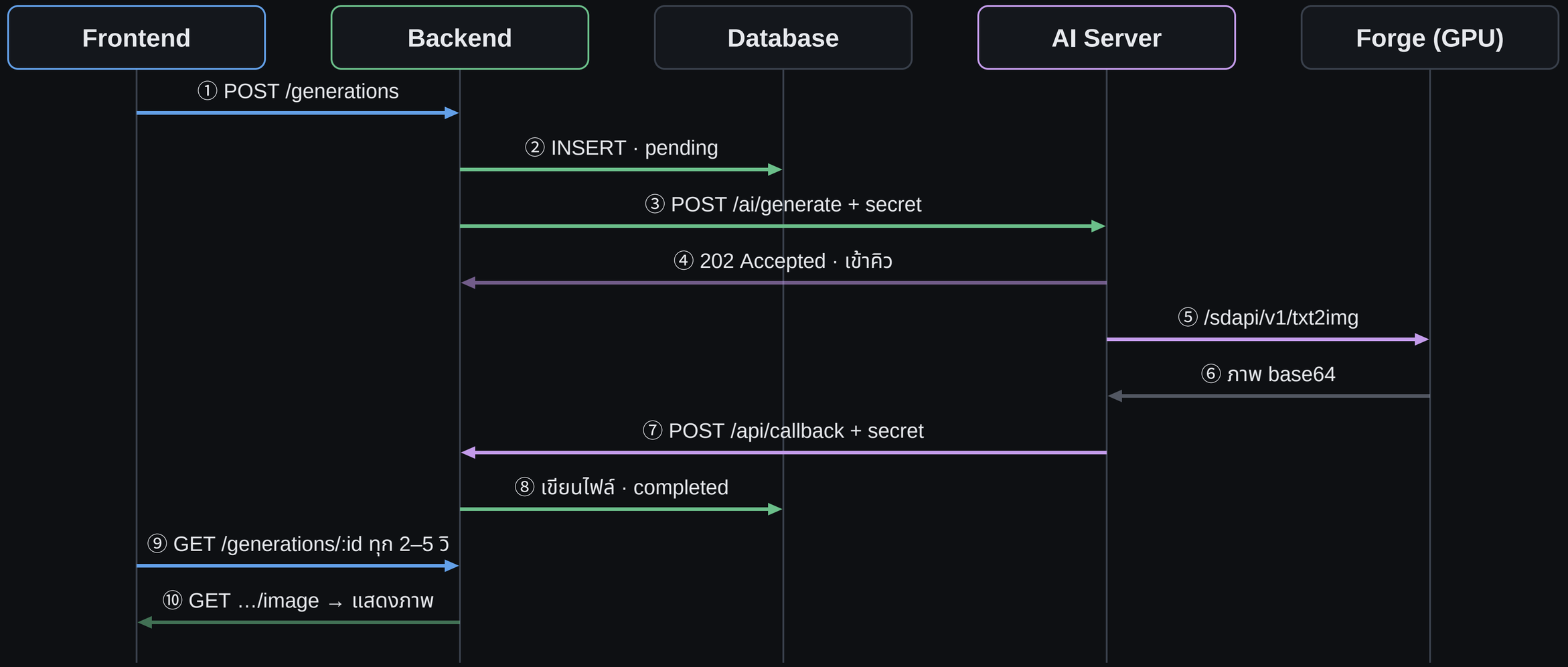
/admin/stats · /users · /audit · /roles

สถาปัตยกรรม 4 โหนดบนวง LAN 10.23.130.x



ผู้ใช้เข้าทาง Nginx จุดเดียว: ไฟล์ .html ไป Node 2, path ของ API ไป Node 3
Backend กับ AI ยืนยันกันด้วย header X-LUMA-INTERNAL-SECRET

วงจรชีวิตของ 1 request สร้างภาพ (โหมด callback)



ลูกศรสีเข้ม = คำขอ · ลูกศรสีจาง = คำตอบ · ทุกคำขอจาก Frontend วิ่งผ่าน Nginx (ละไว้ในภาพ) · สไลด์ถัดไปไล่โค้ดตามลำดับเลขนี้

แผนที่ repo: 1 branch ต่อ 1 โหนด

devops
nginx/

conf/nginx.conf
gateway และการจัดเส้นทาง

test_distributed_system.py
QA 12 ข้อ ข้ามเครื่อง

html/dashboard.html
สถานะ 4 โหนดทุก 5 วิ

backup_system.py
zip + SHA-256

*.bat · *.ps1
เปิด/ปิด/รีโหลด nginx บน Windows

frontend-app
frontend-html/

*.html
7 หน้า: login ถึง admin

js/config.js · api.js
ค่าตั้งต้น + ทุกคำขอไป backend

js/generate/*
หน้าสร้างภาพ แยก 10 ไฟล์

js/mask.js · studio/*
วาด mask, เครื่องมือแต่งภาพ

tests/*.mjs
unit · e2e · regressions · responsive

backend
app/

main.py
รวม 7 router + CORS

api/*.py
auth · generation · upload · models · tools · admin · callback

services/generation.py
หัวใจ: สร้างงาน, ส่งให้ AI, รับผล

core/security.py
bcrypt + JWT

mock_ai_server.py
AI ปลอมไว้ทดสอบเครื่องเดียว

ai-node
ai_server/

server.py
/ai/generate · /ai/edit · /ai/task

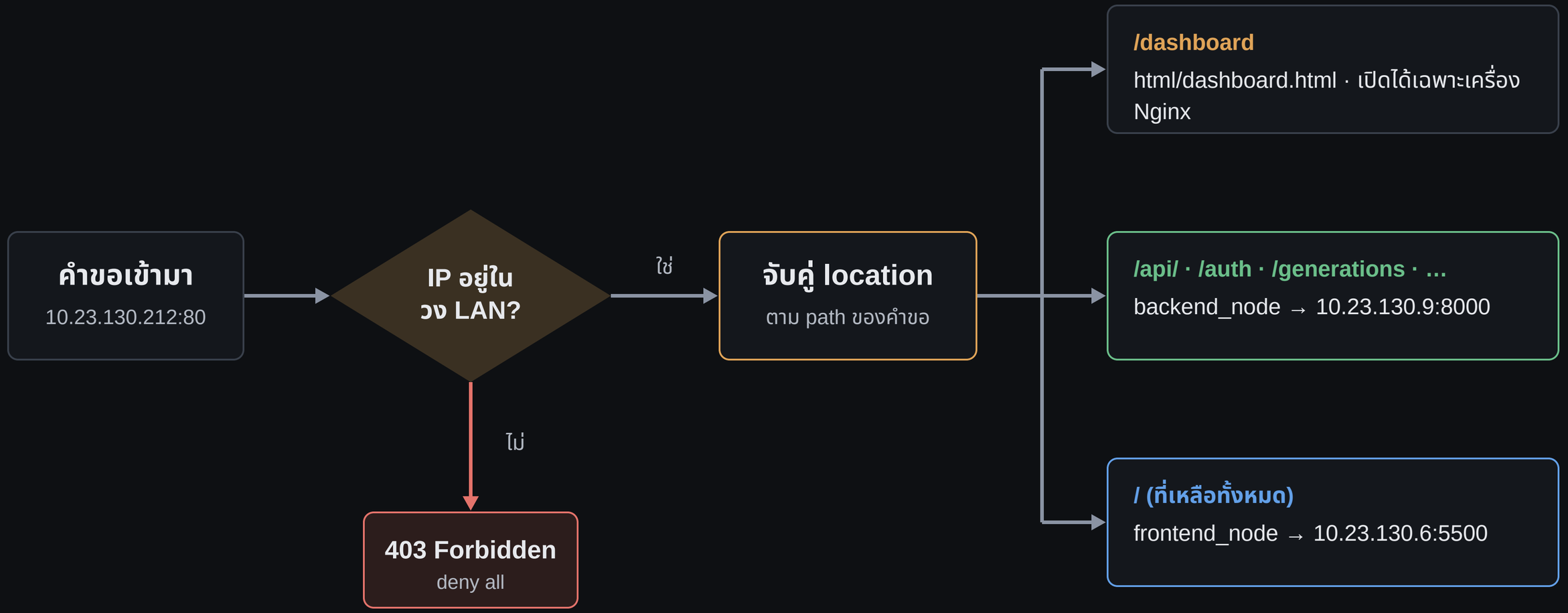
services/queue_manager.py
คิว FIFO + watchdog

services/prompt_builder.py
ใส่ LoRA + กันข้ามตระกูล

services/forge_client.py
คุยกับ Forge :7861

utils/callbacks.py
ส่งผลกลับ ลองใหม่ 3 ครั้ง

Nginx ตัดสินว่าคำขอไปเครื่องไหน



เดิม regex ของ backend จับ /admin.html ไปด้วย · DevOps แก้แล้ว (9dcb2f6) ดูสไลด์ถัดไป

ไล่โค้ด nginx.conf: ด่าน LAN แล้วตามด้วย location 4 อัน

```
45  allow 127.0.0.1;
47  allow 172.20.10.0/24;
50  deny all;
      …
76  location ~* ^/dashboard(\.html)?$ {
80      allow 172.20.10.9;
81      deny all;
      …
91  location /api/ {
92      proxy_pass http://backend_node;
95  location ~ ^/(…|tools|admin|callback|…) {
96      proxy_pass http://backend_node;
      …
102 location / {
103     proxy_pass http://frontend_node;
```

1 · บรรทัด 45–50 ด่านแรกระดับ server: IP นอก LAN โดน 403 ก่อนถึง location ไหนเลย

2 · บรรทัด 76 regex ~* ไม่สนตัวพิมพ์ เปิด dashboard ได้เฉพาะตัวเครื่อง Nginx

3 · บรรทัด 91 prefix /api/ (models, tools, callback) ไป backend

4 · บรรทัด 95 route ของ backend ที่ไม่มี /api นำหน้า เช่น /auth, /generations

5 · บรรทัด 102 ที่เหลือทั้งหมด คือไฟล์หน้าเว็บ ไป frontend

บั๊กที่พบ (แก้แล้วใน devops 9dcb2f6): regex ไม่มีตัวปิดท้าย /admin.html จึงถูกส่งไป backend (ได้ 404)

ที่แก้: location = /admin.html { proxy_pass http://frontend_node; } · และใน regex เปลี่ยน admin เป็น admin/

QA 12 ข้อ ตรวจจากข้างนอกเข้าไปทีละชั้น

กลุ่ม 1

พอร์ตเปิดอยู่ไหม

ต่อ TCP ไปที่ :80, :5500, :8000, :7860 ของแต่ละเครื่อง

กลุ่ม 2

แต่ละเครื่องตอบถูกไหม

/ai/health, /healthz + DB, /login.html

html/dashboard.html

Dashboard

แสดงสถานะ 4 โหนด อัปเดตเองทุก 5 วินาที เปิดได้จากเครื่อง Nginx เท่านั้น

กลุ่ม 3

ผ่าน Nginx แล้วถึงปลายทางไหม

/api/models ได้ 401 ก็ถือว่าผ่าน เพราะแปลว่าไปถึง backend แล้ว ไม่ใช่ 404/502

กลุ่ม 4

บังคับผ่าน gateway

อ่าน config.js ต้องเป็น API_BASE_URL = ''

backup_system.py

Backup

zip config, หน้าเว็บ, ซอร์ส พร้อม checksum SHA-256 และบันทึกสุขภาพระบบ ณ ตอนนั้น

หมายเหตุ: ตาราง QA ใน README ของ devops (เช่น /api/auth/login) ไม่ตรงกับ 12 ข้อในโค้ดจริง ควรอัปเดตให้ตรงกัน

เส้นทางของผู้ใช้ในหน้าเว็บ



ทุกหน้าโหลด config.js → api.js → layout.js ก่อน · token เก็บใน localStorage · ได้ 401 เมื่อไหร่ ลบ token แล้วพากลับหน้า login

api.js: ทุกคำขอไป backend ผ่านฟังก์ชันเดียว

```
 42  function authHeaders() {
 43    const token = getToken();
 44    return token ? { Authorization: 'Bearer ' + token } : {};
       …
 98  async function apiRequest(path, options = {}) {
 99    const headers = options.auth === false ? {} : authHeaders();
102    if (options.json !== undefined) {
103      headers['Content-Type'] = 'application/json';
104      body = JSON.stringify(options.json);
       …
113      response = await fetch(API_BASE_URL + path,
           { method: options.method || 'GET', headers, body });
114    } catch (networkError) {
115      throw new ApiError(0, errorMessage(0));
       …
125      if (response.status === 401 && !options.quiet401) handleUnauthorized();
126      throw new ApiError(response.status, errorMessage(…));
       …
129    if (response.status === 204) return null;
131    return text ? JSON.parse(text) : null;
```

1 · บรรทัด 42–44 แนบ Bearer token จาก localStorage ถ้ายังไม่ล็อกอินก็ไม่ส่ง header เลย

2 · บรรทัด 102–104 ส่ง JSON หรือแบบฟอร์ม (หน้า login ของ FastAPI รับแบบฟอร์ม)

3 · บรรทัด 113–115 เน็ตล่ม → ApiError(0) พร้อมข้อความไทย

4 · บรรทัด 125 401 = token หมดอายุ → ลบ token แล้วพาไปหน้า login

API_BASE_URL = ''

fetch ไปที่ /generations แบบ path เปล่า เบราว์เซอร์จึงยิงกลับไปที่ 10.23.130.212 เดิม ผ่าน Nginx เป็น origin เดียวกัน ไม่ต้องพึ่ง CORS

run.js: กดสร้าง → ถามสถานะ → โหลดภาพ

```
 55  async function submit() {
 63      if (mode === 'inpaint') {
 65        const maskBlob = await MaskEditor.exportMask();
 66        maskUrl = (await apiUpload('/uploads', maskBlob, 'mask.png')).url;
 68      const created = await apiRequest('/generations',
           { method: 'POST', json: buildRequest(maskUrl) });
 69      watchRun(created);
       …
130  async function poll() {
131    const token = watchToken;
134      const latest = await apiRequest('/generations/' + runId);
135      if (token !== watchToken) return;
138        progress = await apiRequest('/generations/' + runId + '/progress')…
146    if (isFinished(run)) return onFinished();
149    if (watched > POLL_GIVE_UP_MS) {   // 5 นาที
156    pollTimer = setTimeout(poll, watched > 60 * 1000 ? 5000 : 2000);
       …
170      const blob = await apiBlob('/generations/' + runId + '/image');
173      runImageUrl = URL.createObjectURL(blob);
```

1 · บรรทัด 55–69 โหมด inpaint ต้องอัปโหลด mask ก่อน แล้ว POST งานด้วย buildRequest() (บรรทัด 29) ที่รวมค่าจากฟอร์มทั้งหมด

2 · บรรทัด 134–138 ถามสถานะงาน และถ้ายังไม่เสร็จก็ถามความคืบหน้าด้วย

3 · บรรทัด 131, 135 watchToken กันคำตอบเก่ามาทับ: ถ้าผู้ใช้เปลี่ยนงานระหว่างรอ คำตอบนั้นถูกทิ้ง

4 · บรรทัด 149, 156 ถามทุก 2 วิในนาทีแรก, ทุก 5 วิ หลังจากนั้น, เลิกถามที่ 5 นาที

5 · บรรทัด 170–173 ภาพต้องใช้ token จึงดึงเป็น blob แล้วทำ object URL และคืนหน่วยความจำเมื่อเลิกแสดง

กรอง LoRA ตามตระกูลโมเดล และสร้าง mask ขาวดำ

js/generate/families.js

```
29  function detectModelFamily(modelId) {
31    if (['pdxl', 'pony'].some((key) => name.includes(key)))
        return 'pony_xl';
32    if (['illu', 'nova', 'xl'].some((key) => name.includes(key)))
        return 'illustrious_xl';
33    return 'sd15';
      …
38    if (model.family) return model.family;
      …
51  function compatibleLoras(loras, family) {
52    return loras.filter((lora) => {
53      const own = loraFamily(lora);
54      return own === 'unknown' || own === family;
```

ใช้ family ที่ AI node ส่งมาก่อน (บรรทัด 38) ไม่มีจึงเดาจากชื่อไฟล์ เปลี่ยนโมเดลแล้ว LoRA ที่ใช้ไม่ได้จะถูกรีเซ็ตเป็น "ไม่ใช้"

js/mask.js · MaskEditor

```
196  exportMask() {
197    const out = document.createElement('canvas');
202    ctx.fillStyle = '#ffffff';
203    ctx.fillRect(0, 0, out.width, out.height);
204    ctx.globalCompositeOperation = 'destination-in';
205    ctx.drawImage(this.canvas, 0, 0);
206    ctx.globalCompositeOperation = 'destination-over';
207    ctx.fillStyle = '#000000';
208    ctx.fillRect(0, 0, out.width, out.height);
210    return new Promise((resolve) => out.toBlob(resolve, 'image/png'));
```

บรรทัด 204 เก็บสีขาวเฉพาะตรงที่ระบาย · บรรทัด 206 เติมดำข้างหลังส่วนที่เหลือ ได้ PNG ขาว = แก้, ดำ = คงไว้ · undo เก็บได้ 3–30 ขั้นตามขนาดภาพ (บรรทัด 139)

Studio: ใช้เครื่องมือต่อกันได้หลายชั้น แล้วส่งไปสร้างภาพต่อ

ภาพต้นฉบับ → POST /api/tools/remove-bg → เวอร์ชัน 2 (parent = 1) + mask → /api/tools/sketch → ส่งไป generate.html

```
28  async function applyTool() {
30    const mySession = session;
36      fields.blur_ksize = detail;
40      fields.target_color = color;
49      const result = await apiUpload('/api/tools/' + tool,
          sourceVersion.blob, 'source.png', fields);
50      const blob = await apiBlob(toServerPath(result.result_image_url));
53      if (mySession !== session) return;
55      const version = { blob, …, tool, detail, parent: sourceVersion };
59      await addVersion(version);
      …
78  async function sendToGenerate(blob, extra, button) {
81      const uploaded = await apiUpload('/uploads', blob, 'studio.png');
82      sessionStorage.setItem('luma.handoff', JSON.stringify({ … }));
90      location.href = 'generate.html';
```

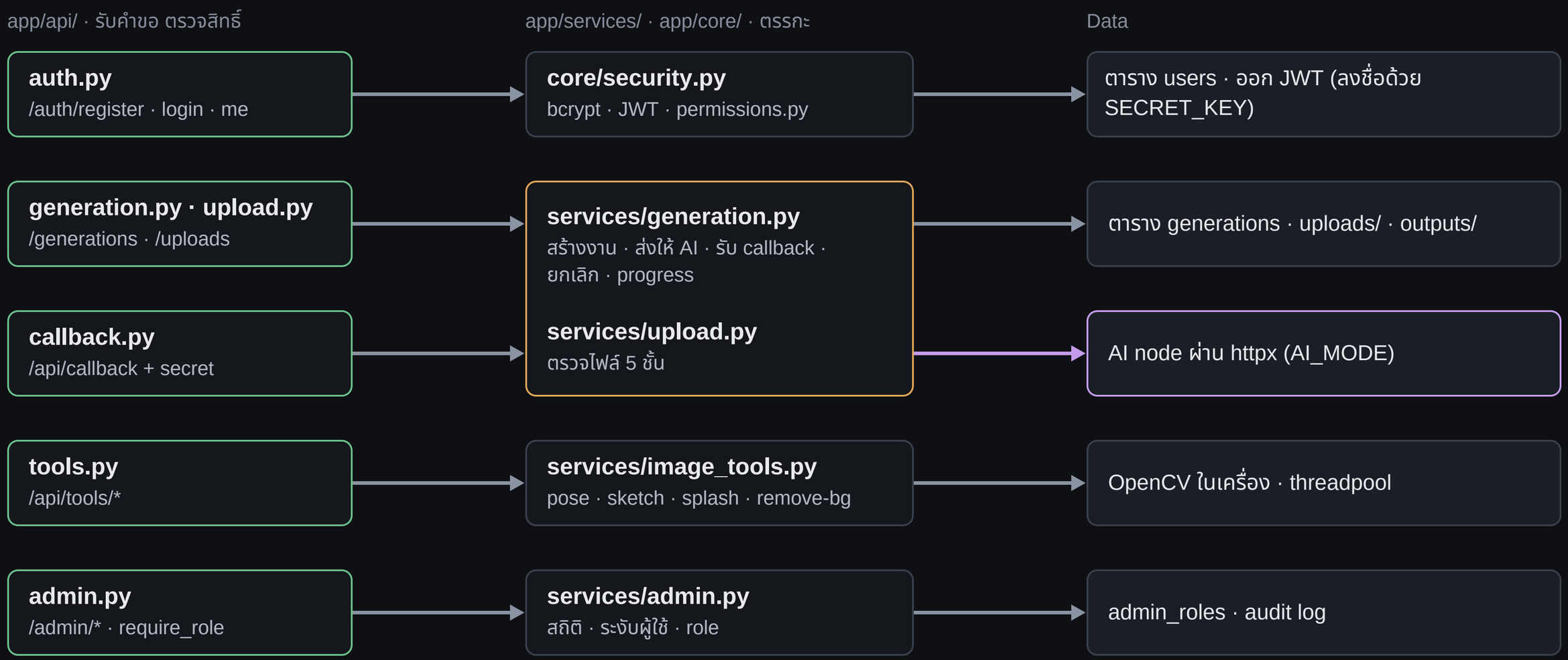
1 · บรรทัด 49 ส่ง "ตัวไฟล์" ไปทุกครั้ง ไม่ส่งที่อยู่ไฟล์ จึงเอาผลของเครื่องมือหนึ่งไปเข้าอีกเครื่องมือได้

2 · บรรทัด 30, 53 session กันผลมาช้าทับภาพใหม่ แบบเดียวกับ watchToken

3 · บรรทัด 55 ทุกผลเป็นเวอร์ชันใหม่ที่ชี้กลับไปหาต้นทาง ย้อนกลับและเทียบก่อน/หลังได้

4 · บรรทัด 78–90 ส่งต่อด้วย sessionStorage หน้า generate อ่านแล้วเปิดเป็น img2img (หรือ inpaint ด้วย mask จาก remove-bg)

Backend แบ่ง 3 ชั้น: router → service → Data



ทุก router ได้ DB session จาก Depends(get_db) และผู้ใช้จาก Depends(get_current_user) · main.py รวม 7 router

ยืนยันตัวตนด้วย bcrypt + JWT แล้วเช็กสิทธิ์ด้วย role

app/core/security.py

```
26  def hash_password(password: str) -> str:
31      salt = bcrypt.gensalt()
32      return bcrypt.hashpw(pwd_bytes, salt).decode('utf-8')
        …
54  def create_access_token(data: dict) -> str:
58      expire = datetime.now(timezone.utc) + timedelta(
            minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
59      to_encode.update({"exp": expire})
62      encoded_jwt = jwt.encode(to_encode, settings.SECRET_KEY, …)
        …
70          payload = jwt.decode(token, settings.SECRET_KEY, …)
103     user_id_str = payload.get("sub")
```

รหัสผ่านเก็บเป็น hash ที่มี salt ในตัว · token มี sub = user id และ exp · jwt.decode ตรวจทั้งลายเซ็นและวันหมดอายุ

app/core/permissions.py

```
19  class Role(IntEnum):
22      REVIEWER = 1
23      ADMIN = 2
24      OWNER = 3
        …
45  def require_role(minimum: Role):
53      def dependency(current_user = Depends(get_current_user), …):
57          role = get_role(db, current_user.id)
58          if role is None or role < minimum:
61              raise HTTPException(
62                  status_code=status.HTTP_403_FORBIDDEN, …)
```

role เป็นตัวเลข จึงเทียบด้วย < ได้ตรง ๆ · ติดที่ระดับ router (admin.py บรรทัด 34) endpoint ใหม่จึงถูกป้องกันอัตโนมัติ · สิทธิ์ไม่ถึงได้ 403 ไม่ใช่ 404

backend:app/core/security.py · app/core/permissions.py · บางบรรทัดย่อพารามิเตอร์ด้วย …

save_uploaded_image(): ไฟล์ต้องผ่าน 5 ด่านก่อนลงดิสก์

1	**Content-Type** png / jpeg / webp เท่านั้น	415
2	**ขนาดไฟล์** ไม่เกิน 10 MB	413
3	**Magic bytes** อ่านหัวไฟล์จริง กันไฟล์ปลอมนามสกุล	422
4	**ถอดรหัสภาพจริง** กัน decompression bomb · ด้านละไม่เกิน MAX_IMAGE_DIMENSION	422
5	**ตัด EXIF + เขียนแบบ atomic** บันทึกใหม่ผ่าน PIL ลง .tmp แล้ว replace	201

```
 29  def validate_magic_bytes(content: bytes) -> str:
 41      if content.startswith(b"\x89PNG\r\n\x1a\n"): return ".png"
 45      if content.startswith(b"\xff\xd8\xff"): return ".jpg"
       …
 58  async def save_uploaded_image(file: UploadFile):
 66      if content_type not in ALLOWED_MIME_TYPES:          # 1
 79      if file_size > settings.MAX_UPLOAD_SIZE_BYTES:      # 2
 94      ext = validate_magic_bytes(content)                 # 3
101          img = Image.open(image_stream)                  # 4
102          img.load()
105      except Image.DecompressionBombError:
117      if width > settings.MAX_IMAGE_DIMENSION or …:
134      temp_path = target_path.with_suffix(".tmp")         # 5
143          save_img.save(clean_buffer, format=img.format or "PNG")
144          temp_path.write_bytes(clean_buffer.getvalue())
145          temp_path.replace(target_path)
```

ด่านไหนไม่ผ่านจะหยุดทันทีและตอบรหัส HTTP ตามคอลัมน์ขวา · ผ่านครบ 5 ด่านแล้วจึงได้ 201 พร้อม path ของไฟล์

backend:app/services/upload.py · เลข # 1–5 ท้ายบรรทัดใส่เพิ่มเพื่อจับคู่กับด่าน · 41/45 ย่อ if + return เป็นบรรทัดเดียว

POST /generations: ตอบทันที แล้วส่งงานให้ AI เบื้องหลัง

app/api/generation.py

```
35      new_job = generation_service.create_generation_job(…)
42      background_tasks.add_task(
43          generation_service.process_generation_task,
44          generation_id=new_job.id)
48      return GenerationResponse.model_validate(new_job)
```

app/services/generation.py

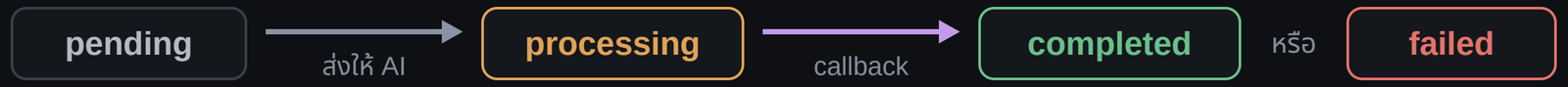
```
406 def create_generation_job(db, user_id, data) -> Generation:
420         detail=f"task_type '…' requires 'source_image_path'"
431         status=GenerationStatus.PENDING.value,
449     db.commit()
      …
485 async def process_generation_task(generation_id):
505         generation.status = GenerationStatus.PROCESSING.value
539         if settings.AI_MODE.lower() == "callback":
549             edit_endpoint = …replace("/ai/generate", "/ai/edit")
566                 "callback_url": settings.BACKEND_CALLBACK_URL,
590                 res = await client.post(target_url, json=…, headers=…)
598                 _mark_as_failed(generation, f"Failed to submit …")
623         target_url = settings.AI_SERVER_DIRECT_URL or settings.AI_SERVER_URL
```

1. 35–48 · บันทึกงาน pending แล้วตอบ 201 ทันที งานจริงไปทำใน BackgroundTasks
2. 420 · img2img / inpaint ต้องมีภาพต้นฉบับอยู่จริงบนเครื่อง ไม่งั้น 422/404
3. 539–598 · โหมด callback: txt2img ไป /ai/generate, img2img/inpaint ไป /ai/edit พร้อมภาพ base64 · ส่งไม่ติดก็ failed ทันที
4. 623 · โหมด direct: รอภาพตอบกลับใน request เดียว ใช้กับ mock_ai_server.py ตอนทดสอบ

ค่าเริ่มต้นใน .env.example คือ AI_MODE=direct ชี้ไป 10.23.130.112:8001 · ใช้งานจริงข้ามเครื่องต้องตั้ง callback + :7860

backend:app/api/generation.py · app/services/generation.py · … = ย่อพารามิเตอร์

process_ai_callback(): รับภาพจาก AI แล้วปิดงาน



app/api/callback.py

```
 26      if not x_luma_internal_secret or … != expected_secret:
 29              status_code=status.HTTP_403_FORBIDDEN,
```

app/services/generation.py

```
701      stmt = select(Generation).where(Generation.id == payload.task_id)
705      if not generation:                          → 404
715      if generation.status == GenerationStatus.COMPLETED.value:
             return 200, …(duplicate=True)
726      if payload.status.lower() == "failed":      → เก็บ error_message
743      if not payload.image_base64:                → 422
753          image_data = _safe_b64decode(payload.image_base64)
771      temp_path = file_path.with_suffix(".tmp")
772      temp_path.write_bytes(image_data)
773      temp_path.replace(file_path)
776      generation.status = GenerationStatus.COMPLETED.value
777      generation.output_path = str(file_path.resolve())
788      db.commit()
```

1. 26 · ด่านแรก: header secret ต้องตรงกับ AI_CALLBACK_SECRET ไม่งั้น 403
2. 715 · idempotent: AI ส่งซ้ำ (ตอน retry) ก็ตอบ 200 ไม่เขียนทับ
3. 726–753 · แยกกรณี failed, ไม่มีภาพ, base64 เสีย
4. 771–773 · เขียน .tmp แล้ว replace: ไฟล์ปลายทางไม่มีวันครึ่ง ๆ กลาง ๆ · แยกโฟลเดอร์ตาม user
5. 776–788 · ตั้ง completed + path ของไฟล์ รอบหน้าที่หน้าเว็บถามก็เห็นว่าเสร็จ

backend:app/api/callback.py · app/services/generation.py · "→ …" คือผลของบรรทัดนั้น ไม่ใช่โค้ด

Studio tools: งานหนัก CPU ต้องไม่ทำให้ API ค้าง

```
34  _tool_slots = asyncio.Semaphore(2)
37  async def _run_in_pool(fn, *args, **kwargs):
39      async with _tool_slots:
40          return await asyncio.wait_for(
41              run_in_threadpool(fn, *args, **kwargs),
42              timeout=60.0
43          )
      …
158     blur_ksize: int = Form(21, ge=3, le=51, …),
```

1. 34 · ทำพร้อมกันได้สูงสุด 2 งาน ที่เหลือรอคิว
2. 41 · โยน OpenCV ไปทำใน thread แยก event loop จึงยังตอบ /healthz ได้ในไม่กี่ ms
3. 42 · เกิน 60 วิ ตัดทิ้ง · 158 ค่าผิดช่วงได้ 422 จาก FastAPI เอง

POST /api/tools/pose
หาโครงกระดูกท่าทาง หลายรอบปรับเอง คืน landmarks + เตือนถ้าไม่เจอคน

POST /api/tools/sketch
ภาพลายเส้นดินสอ ปรับความเบลอ 3–51

POST /api/tools/color-splash
เก็บสีเดียว (green / red) ที่เหลือเป็นขาวดำ

POST /api/tools/remove-bg
ตัดพื้นหลัง: MediaPipe (คน) หรือ GrabCut (สิ่งของ) คืนภาพ + mask ให้ส่งต่อไป inpaint

ลายเส้น และ เน้นสี: OpenCV ล้วน ไม่ใช้ AI

ลายเส้น · /api/tools/sketch

Color Dodge Blend

1. Gray = cvtColor(BGR → GRAY)
2. Invert = 255 − Gray
3. Blur = GaussianBlur(Invert, k × k)
4. Sketch = Gray ÷ (255 − Blur) × 256

ทำไมได้ลายเส้น

- พื้นที่เรียบ: เบลอแล้วค่าแทบไม่เปลี่ยน → ผลหาร ≈ 255 → ขาว
- ขอบวัตถุ: ค่าต่างกันมาก → เหลือเป็นเส้นเข้ม
- แถบ "ความนุ่มของเส้น" = k (เลขคี่ 3–51) ยิ่งมากเส้นยิ่งหนาและนุ่ม

เน้นสี · /api/tools/color-splash

กรองสีใน HSV

1. HSV = cvtColor(BGR → HSV)
2. mask = inRange(HSV, ช่วงสีที่เลือก)
3. ส่วนสี = ภาพเดิม AND mask
4. ผล = ส่วนสี + (ขาวดำ AND NOT mask)

ทำไมต้อง HSV

- H คือเฉดสี แยกจากความสว่าง → เลือกสีได้แม่นกว่า RGB
- เขียว: H 35–85
- แดง: H 0–10 รวมกับ 170–180 เพราะแดงอยู่ตรงรอยต่อของวงล้อสี

ท่าทาง และ ตัดฉาก: หาคน และแยกคนออกจากฉาก

ท่าทาง · /api/tools/pose

MediaPipe Pose · 33 จุดบนร่างกาย

1. รอบ 1: complexity 1 · มั่นใจ 0.5 (ภาพคนจริง)
2. รอบ 2: complexity 1 · 0.3 (ภาพวาด / อนิเมะ / AI)
3. รอบ 3: complexity 2 · 0.25 (ท่ายาก)
4. วาดโครงกระดูก + คืน x, y, z, visibility

จุดที่ควรรู้

- เจอคนรอบไหนหยุดทันที ไม่ต้องรันครบ 3 รอบ
- ไม่เจอคน → landmarks = [] หน้าเว็บขึ้น "ไม่พบคนในภาพนี้"
- หน้าเว็บกรองจุดตาม visibility ได้ (renderPose)

ตัดฉาก · /api/tools/remove-bg

Hybrid: MediaPipe Selfie Seg → GrabCut สำรอง

1. MediaPipe Selfie Seg: ค่า > 0.45 = คน
2. เจอคน ≥ 0.5% ของภาพ → ใช้ mask นี้
3. ไม่เจอคน → GrabCut: ย่อ ≤ 512 px · เว้นขอบ 5%
4. MORPH_CLOSE 5×5: อุดรู เกลี่ยขอบ
5. รวม mask กับภาพ → ภาพโปร่งใส + mask

จุดที่ควรรู้

- mask (ขาว = ตัวแบบ) ส่งต่อไป inpaint วาดฉากหลังหรือตัวแบบใหม่ได้
- คน → MediaPipe · สิ่งของ → GrabCut

AI Node: End-to-End Flow 6 ขั้น

Backend → POST /ai/generate · /ai/edit พร้อม header X-LUMA-INTERNAL-SECRET

1 รับงาน + ตรวจ secret
server.py
ผิด → 403 · ถูก → เข้าคิวแล้วตอบ 202 ทันที ไม่รอ GPU

→

2 คิว FIFO + Watchdog
services/queue_manager.py
GPU มี VRAM 8 GB จึงทำทีละงาน · คุมเวลางานละ 120 วิ

→

3 LoRA Guard
services/prompt_builder.py
ตระกูลไม่ตรง → ตัด LoRA ทิ้ง · เติม <lora> + trigger word

↓

4 ปรับขนาดภาพต้นฉบับ
utils/image_utils.py
เฉพาะ img2img / inpaint: 256–768 px และหาร 8 ลงตัว

←

5 Forge API (GPU)
services/forge_client.py
127.0.0.1:7861 · txt2img / img2img (+ mask) → WebP 92

←

6 Callback + ล้าง VRAM
utils/callbacks.py · gpu_monitor.py
ส่งผลกลับ Backend ได้ 3 ครั้ง เว้น 1, 2 วิ · ล้าง VRAM ทุกงาน

↑ ผลลัพธ์กลับไปที่ Backend: POST /api/callback

เส้นทางผิดพลาด
secret ผิด → 403 ตั้งแต่ขั้น 1 · เกิน 120 วิหรือ error → interrupt Forge แล้ว callback status = failed · ล้าง VRAM ใน finally ทุกกรณี

งานที่ถูกยกเลิกระหว่างรอคิว (DELETE /ai/task/{id}) ถูกข้ามตอนถึงคิว · Backend ได้ callback เสมอ ทั้งสำเร็จ ล้มเหลว และถูกยกเลิก

รับงานแล้วตอบ 202 ทันที → คิว FIFO ทำทีละงาน

```
     server.py
590  @app.post("/ai/generate", status_code=status.HTTP_202_ACCEPTED)
591  async def generate_image(req: GenerateRequest,
593          x_luma_internal_secret: Optional[str] = Header(None)):
600      verify_internal_secret(x_luma_internal_secret)
601      q_pos = await task_queue.enqueue(req.model_dump(),
                                          handle_txt2img_inference)
602      return {"status": "accepted", "queue_position": q_pos, …}

     services/queue_manager.py · _process_queue()
224  task_data, handler = await self._queue.get()
230  if state_info.get("status") == "cancelled": continue
245      task_coro = asyncio.to_thread(handler, task_data)
249      result = await asyncio.wait_for(task_coro,
251          timeout=AIConfig.TASK_TIMEOUT_SECONDS)    # 120
267      await send_callback_with_retry(status="completed", …)
280  except asyncio.TimeoutError:
284      await asyncio.to_thread(interrupt_forge_generation)
285      await send_callback_with_retry(status="failed", …)
302  finally:
304      clear_vram_cache()
308      self._queue.task_done()
```

1. server.py 600 · ด่านแรก: secret ใน header ไม่ตรง → 403 คนนอกยิงงานเข้า GPU ไม่ได้
2. 601–602 · ใส่คิวแล้วตอบ 202 + ลำดับคิวทันที ไม่รอ GPU เจนเสร็จ → Nginx / Backend ไม่เจอ 504 Timeout
3. 245–251 · ทำทีละงาน เพราะ RTX 3070 มี VRAM 8 GB รัน 2 งานพร้อมกันจะ OOM · to_thread ทำให้ event loop ยังตอบ /ai/health ได้
4. 280–285 · เกิน 120 วิ → interrupt Forge แล้วส่ง failed งานถัดไปจะไม่ชนกับงานค้าง
5. 302–304 · finally: ล้าง VRAM ทุกครั้ง ทั้งงานสำเร็จและล้มเหลว (ขั้น 6)

ai-node:ai_server/server.py · services/queue_manager.py · … = ย่อ · ตัดย่อหน้าบางบรรทัดออก

LoRA Guard → ปรับขนาดภาพก่อนเข้า GPU

services/prompt_builder.py · ขั้น 3

```
62  def is_lora_compatible(lora_id, model_name=None) -> bool:
66      if not lora_id or lora_id not in LORA_REGISTRY or not model_name:
67          return True
70      if not lora_family or lora_family == "unknown": return True
72      return lora_family == detect_model_family(model_name)
75  def build_prompt_with_lora(user_prompt, lora_id=None, model_name=None):
92      if model_name and not is_lora_compatible(lora_id, model_name):
100         return user_prompt.strip(), ""      # ตัด LoRA ทิ้ง
111     lora_tag = f"<lora:{lora_name}:{weight}>"
116     for phrase in trigger.split(","):
117         if phrase.lower() not in user_prompt.lower():
118             words_to_add.append(phrase)
132     return final_prompt, lora_tag
```

utils/image_utils.py · ขั้น 4

```
54  def enforce_max_resolution(image, min_dim=256, max_dim=768):
74      if w < min_dim or h < min_dim:      # เล็กเกิน → ขยาย
75          ratio = max(min_dim / w, min_dim / h)
79      elif w > max_dim or h > max_dim:    # ใหญ่เกิน → ย่อ
80          ratio = min(max_dim / w, max_dim / h)
85      new_w = max((w // 8) * 8, min_dim)
86      new_h = max((h // 8) * 8, min_dim)
92      return image.resize((new_w, new_h), Image.Resampling.LANCZOS)
```

1. 62–72 · ตระกูล LoRA ต้องตรงกับโมเดล (sd15 / pony_xl / illustrious_xl) · ไม่อยู่ใน registry หรือ unknown = ผ่าน
2. 92–100 · ไม่ตรง → ตัด LoRA ทิ้งแล้วสร้างภาพต่อ ไม่ error (ด่านที่ 2 ต่อจากตัวกรองหน้าเว็บ)
3. 111–118 · Frontend ส่งมาแค่ lora_id · AI ใส่ <lora:ชื่อ:น้ำหนัก> และ trigger word ที่ผู้ใช้ยังไม่ได้พิมพ์
4. 74–82 · เฉพาะ img2img / inpaint: ย่อหรือขยายภาพต้นฉบับให้อยู่ใน 256–768 px รักษาสัดส่วน
5. 85–92 · ปัดให้หาร 8 ลงตัว เพราะ VAE บีบภาพลง latent 8 เท่า (f = 8) · resize ด้วย Lanczos เก็บขอบคม

Forge สร้างภาพ → ส่ง WebP กลับ + ล้าง VRAM

services/forge_client.py · ขั้น 5

```
152  def run_txt2img(prompt, negative_prompt, steps, …):
170      if checkpoint: set_forge_model(checkpoint)
176      payload = {"prompt": …, "steps": min(max(steps, 1), 50),
181          "width": min(max(width, 256), 768), "seed": …}
191      resp = requests.post(
             f"{FORGE_API_URL}/sdapi/v1/txt2img",
             json=payload, timeout=120.0)
197      pil_img = decode_base64_to_image(images[0])
198      webp_b64 = encode_image_to_base64(pil_img, "WEBP")
201      return webp_b64, actual_seed
259  # run_img2img / run_inpaint → /sdapi/v1/img2img
```

> Forge อยู่ที่ 127.0.0.1:7861 ในเครื่อง AI เท่านั้น เปิดออกนอกแค่ :7860 · img2img / inpaint ใช้ /sdapi/v1/img2img (+ mask) · แปลง PNG → WebP คุณภาพ 92 ไฟล์เล็กลง ส่งกลับเร็วขึ้น

ALLOW_FALLBACK_RENDER = true (ค่าเริ่มต้น · config.py 116) ถ้า Forge ล่ม จะได้ภาพสำรองแทน error · ควรปิดตอนเดโม (สไลด์ 25 ข้อ 5)

utils/callbacks.py · gpu_monitor.py · ขั้น 6

```
     utils/callbacks.py · send_callback_with_retry()
 62  headers = {"X-LUMA-INTERNAL-SECRET": …}
 73  cache_file = f"{CACHE_DIR}/{task_id}.webp.b64"
 91  delays = [1.0, 2.0, 4.0]
 93  for attempt, delay in enumerate(delays, start=1):
 96      response = await client.post(callback_url, …)
 99      if response.status_code in [200, 201, 204]:
101          return True
109      if attempt < len(delays):
110          await asyncio.sleep(delay)
     utils/gpu_monitor.py · clear_vram_cache()
 69  gc.collect()
 71  torch.cuda.empty_cache()
 72  torch.cuda.ipc_collect()
```

> ส่งได้ 3 ครั้ง เว้น 1 และ 2 วิ (4.0 ไม่ถูกใช้ เพราะครั้งที่ 3 ไม่ต้องรอ) · ส่งซ้ำปลอดภัยเพราะ Backend เป็น idempotent (สไลด์ 18) · เก็บภาพสำรองในเครื่อง AI ก่อนส่ง · ล้าง VRAM ทุกงาน กัน OOM

ความปลอดภัยเป็นชั้น ๆ จากนอกเข้าใน

เครือข่าย	Nginx รับเฉพาะ IP ในวง LAN · dashboard เปิดได้จากเครื่อง Nginx เท่านั้น · frontend :5500 รับเฉพาะ IP ของ Nginx	nginx.conf · serve_gateway.py*
เบราว์เซอร์	CSP ทุกหน้า: สคริปต์มาจากเว็บเราเท่านั้น ไม่มี inline script	*.html <meta CSP>
ตัวตนและสิทธิ์	bcrypt + salt · JWT มีวันหมดอายุ · role reviewer < admin < owner ติดที่ router	security.py · permissions.py
ข้อมูลขาเข้า	ไฟล์ผ่าน 5 ด่าน · schema จำกัด steps/CFG/ขนาด · เครื่องมือจำกัดช่วงค่าและ 10 MB	upload.py · schemas/ · tools.py
ระหว่างเครื่อง	Backend ↔ AI ใช้ X-LUMA-INTERNAL-SECRET ทั้งขาไปและขากลับ · secret ที่เคยหลุดถูกเปลี่ยนเป็น placeholder	server.py · callback.py
ไฟล์ที่เก็บ	ตัด EXIF (GPS) · เขียนแบบ atomic · ภาพผลลัพธ์แยกโฟลเดอร์ตามผู้ใช้ · ดึงภาพต้องมี token	upload.py · generation.py

* serve_gateway.py อยู่ใน frontend-app แล้ว (90e129f) · Backend :8000 ยังเข้าตรงจาก LAN ได้

จุดที่ต่อกันไม่ตรง ต้องแก้ก่อนรวมโค้ด

#	ปัญหา	ฝ่าย	วิธีแก้
1	หน้า /admin.html ผ่าน Nginx ได้ 404	**DevOps**	แก้แล้ว (9dcb2f6) · ส่ง /admin.html ไป frontend
2	ค่าเริ่มต้นชี้ mock AI (direct :8001) AI จริงไม่มี /generate	**Backend**	AI_MODE=callback + URL ไป 10.23.130.112:7860
3	callback ตั้งไว้วง 192.168.1.x แต่วงจริงคือ 10.23.130.x	**AI**	ตั้ง BACKEND_CALLBACK_URL + secret ให้ตรงกัน
4	README ขัดกัน: 3 กับ 4 โหนด, IP, ตาราง QA	**ทุกฝ่าย**	ตกลงฉบับเดียวแล้วอัปเดตพร้อมกัน
5	ALLOW_FALLBACK_RENDER=true Forge ล่มจะได้ภาพสำรองแทน error	**AI**	ปิดตอนเดโม หรือแสดงให้ชัดว่าเป็นภาพสำรอง
6	LoRA ที่ไม่อยู่ใน registry: guard ปล่อยผ่าน แต่หน้าเว็บซ่อน	**AI · FE**	ให้ /ai/models ส่ง unknown

แผนรวมโค้ด (เสนอ ยังไม่ยืนยัน)

backend → ai-node → frontend-app ลง integration-stage ทดสอบครบแล้วค่อยเข้า main · devops ยังไม่อยู่ในแผน (PR #1 ยังเปิดอยู่)

ที่มา: LUMA-FLOW-AUDIT.md · ai-node:MERGE_PLAN.md

Test case 25 ข้อ ไล่ตามลำดับเดโม ผ่านทุกชุด

1 · เข้าระบบ

- หน้าแรกพาไปล็อกอิน
- รหัสผิด / ล็อกอินสำเร็จ
- สมัครบัญชีใหม่

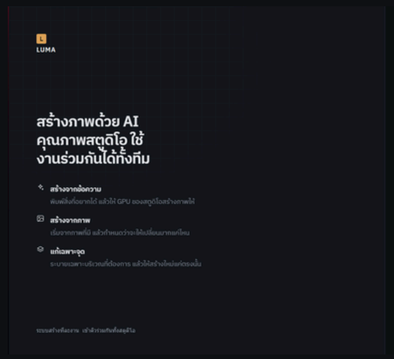
2 · ผู้ใช้ทั่วไป

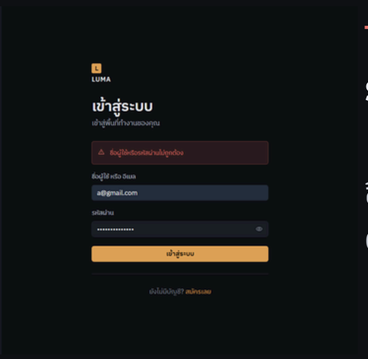
- สร้างภาพ 3 โหมด
- อัปโหลดผิดชนิด
- ประวัติ: ใช้ค่าเดิม / ลบ
- สตูดิโอ 4 เครื่องมือ / บัญชี

3 · แอดมิน

- ผู้ใช้ทั่วไปเข้าไม่ได้ (403)
- Overview / Users
- ปิดบัญชีพร้อมเหตุผล → Audit
- ให้ / ถอนสิทธิ์

เข้าระบบ: ล็อกอิน สมัคร และ token หมดอายุ

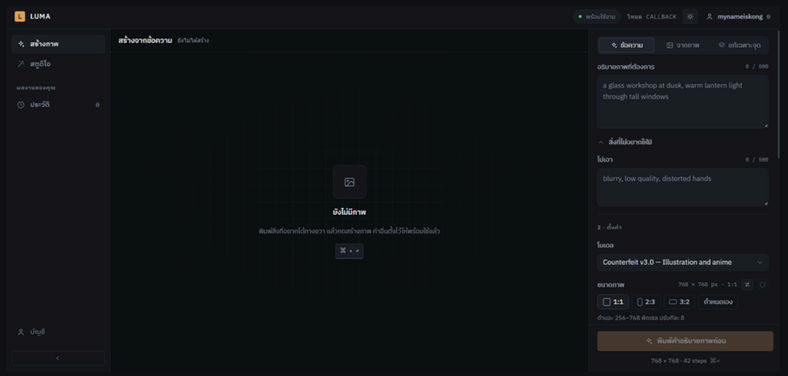




TC-01 · กรณีผิดพลาด

รหัสผ่านผิด

ขึ้น "ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง" และยังอยู่หน้าเดิม

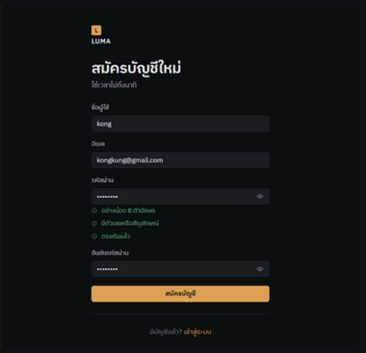


TC-02

ล็อกอินสำเร็จ (owner)

เข้าหน้าสร้างภาพ ไฟสถานะ "พร้อมใช้งาน" และมีลิงก์แผงแอดมิน



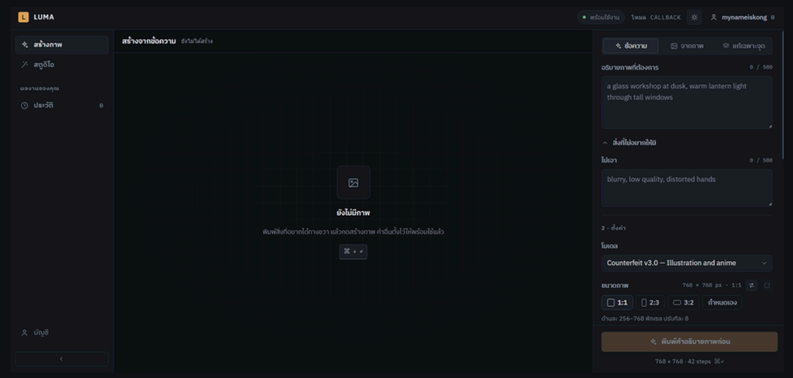


TC-03

สมัครบัญชีใหม่

เงื่อนไขรหัสติ๊กครบ 3 ข้อ ปุ่มสมัครจึงกดได้ สมัครแล้วเข้าระบบให้ทันที

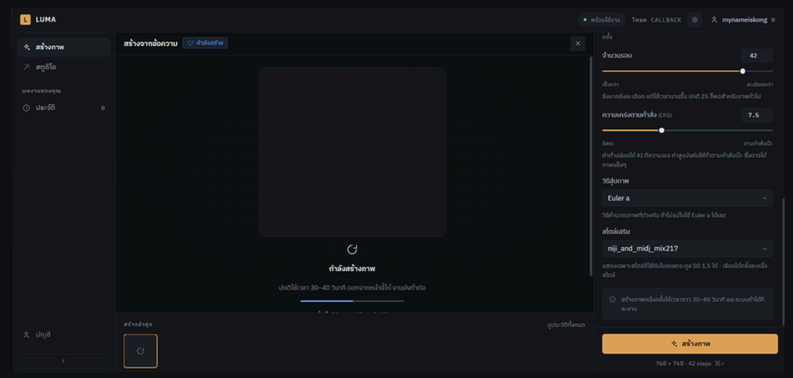
สร้างภาพจากข้อความ และการตรวจค่าก่อนส่ง



TC-04 · กรณีผิดพลาด

ยังไม่มีคำอธิบาย

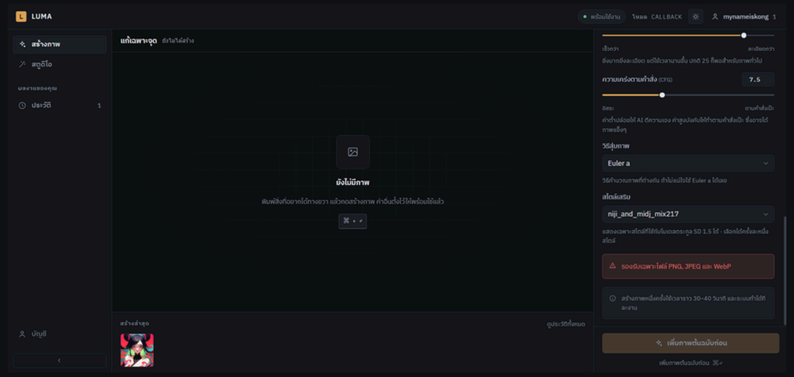
ปุ่มสร้างกดไม่ได้ ขึ้นข้อความ "พิมพ์คำอธิบายภาพก่อน"



TC-05

สร้างภาพจากข้อความ

สถานะ "เสร็จแล้ว" ภาพ 768 × 768 พร้อมแถบแต่งด่วน และงานใหม่ในแถบงานล่าสุด

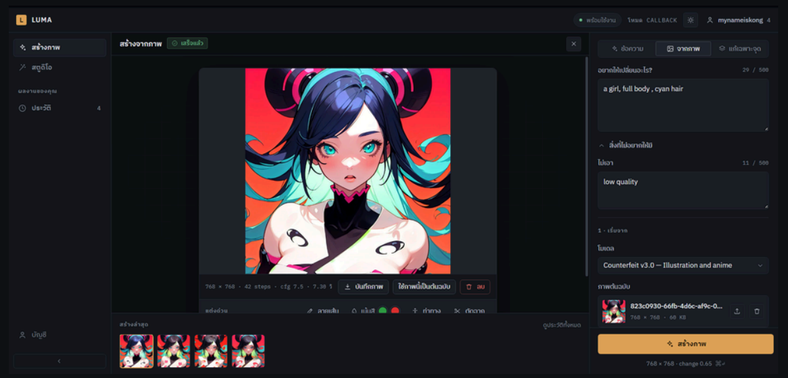


TC-06 · กรณีผิดพลาด

อัปโหลดไฟล์ผิดชนิด

ขึ้น "รองรับเฉพาะไฟล์ PNG, JPEG และ WebP" และไม่ส่งไฟล์ไป backend

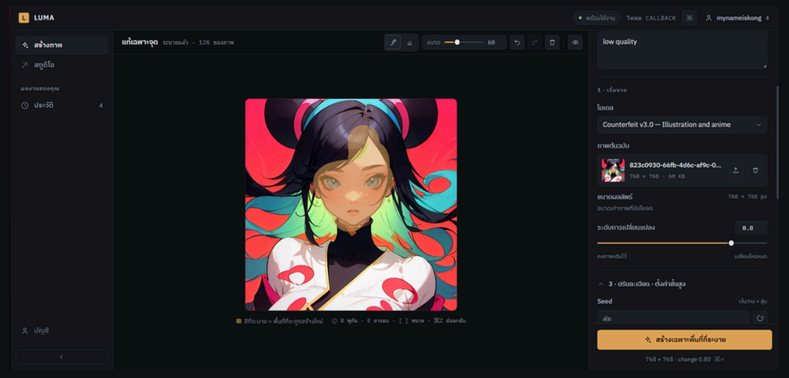
สร้างจากภาพ แก้เฉพาะจุด และแต่งด่วน



TC-07

img2img

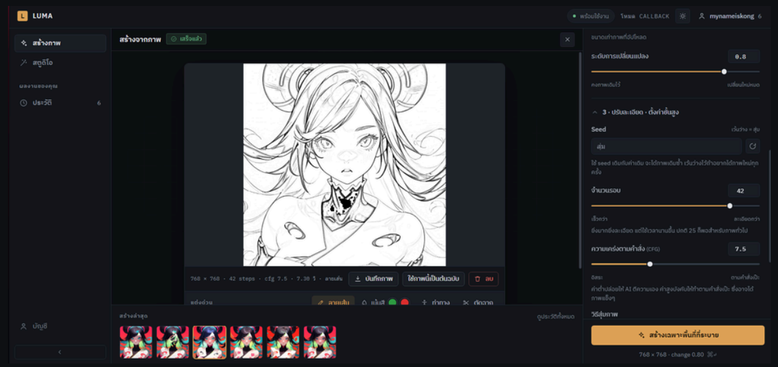
อัปโหลด sample.png 768 x 768 ขนาดผลลัพธ์ปรับตามภาพต้นฉบับ แล้วสร้างจนเสร็จ



TC-08

ระบาย mask

บริเวณสีส้มคือส่วนที่จะให้ AI วาดใหม่ มีปุ่มย้อนกลับ/ทำซ้ำแถบบน

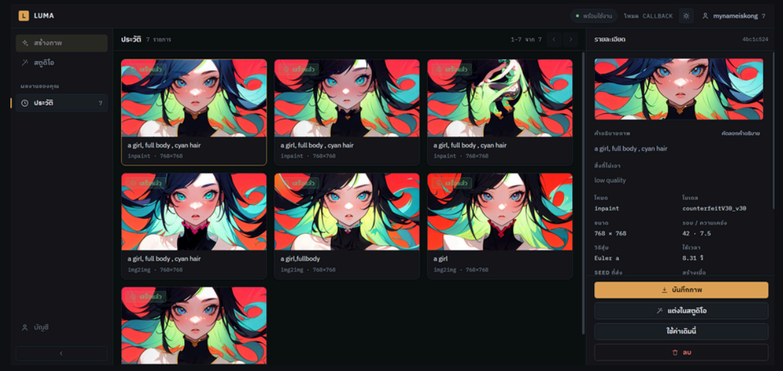


TC-09

แต่งด่วน: ลายเส้น

ภาพผลลัพธ์กลายเป็นลายเส้นดินสอทันที มีปุ่ม "กลับไปภาพเดิม"

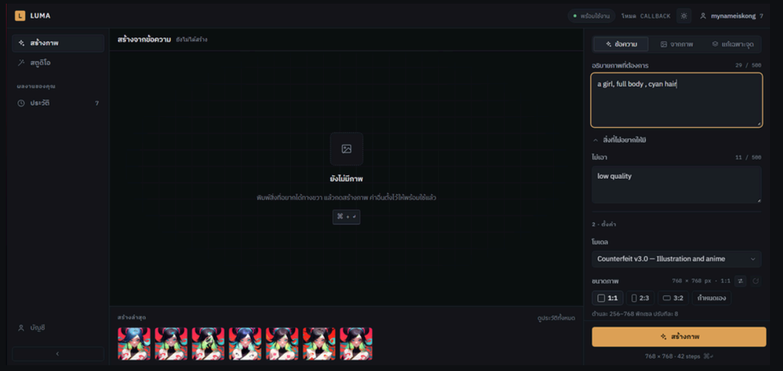
ประวัติ: ดู ใช้ค่าเดิม ลบ และหน้าบัญชี



TC-10

รายละเอียดงาน

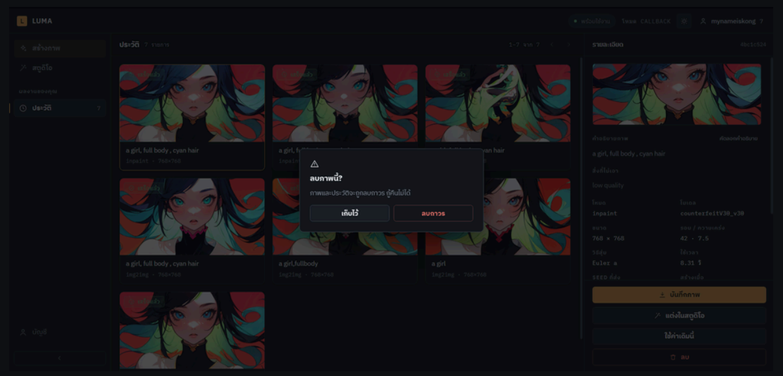
ค่าทั้งหมดของงาน + ภาพต้นฉบับ ลิงก์ ?id= ส่งต่อให้คนอื่นเปิดงานเดียวกันได้



TC-11

"ใช้ค่าเดิม"

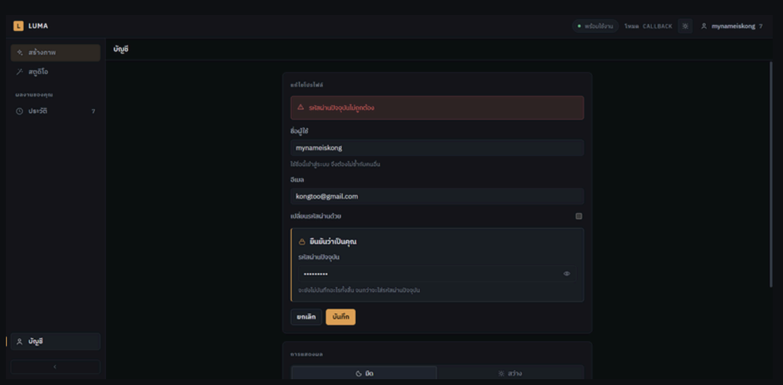
กลับหน้าสร้างภาพพร้อมคำอธิบายและค่าเดิม เปิดเป็นโหมดข้อความเสมอ



TC-12 · กรณีผิดพลาด

ลบต้องยืนยัน

หน้าต่าง "ลบภาพนี้?" เลือกเก็บไว้หรือลบถาวร กดพลาดก็ไม่หาย

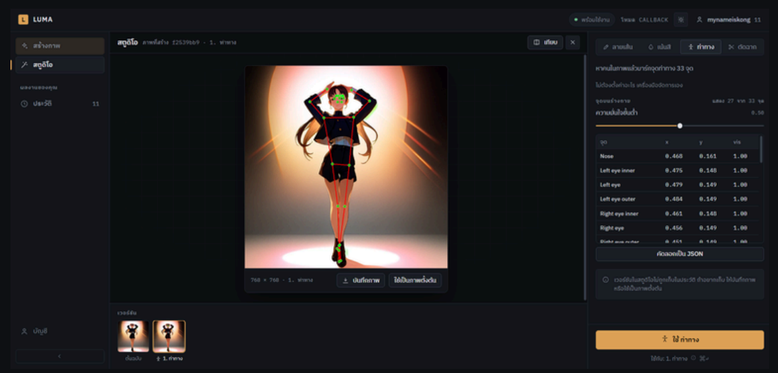


TC-13 · กรณีผิดพลาด

รหัสปัจจุบันผิด

ขึ้น "รหัสผ่านปัจจุบันไม่ถูกต้อง" แต่ยังล็อกอินอยู่ ไม่ถูกเด้งออก

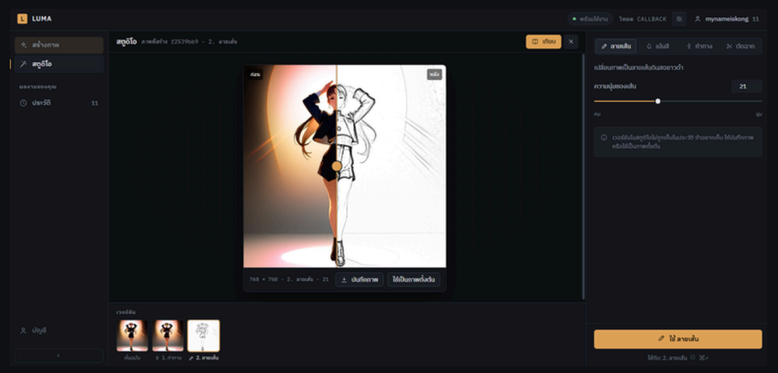
สตูดิโอ: ใช้เครื่องมือต่อกัน แล้วส่งไปวาดใหม่



TC-14

ท่าทาง (pose)

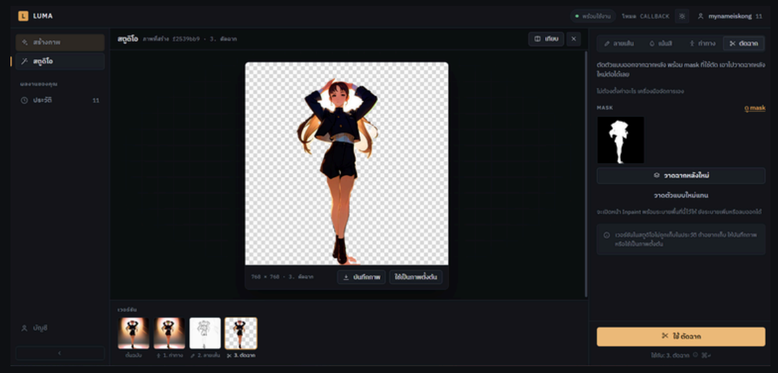
ภาพวาดเรียบ ๆ จึงแจ้งว่าไม่พบคน ใช้ภาพคนจริงตอนเดโมจะเห็นโครงกระดูก



TC-15

เทียบก่อน/หลัง

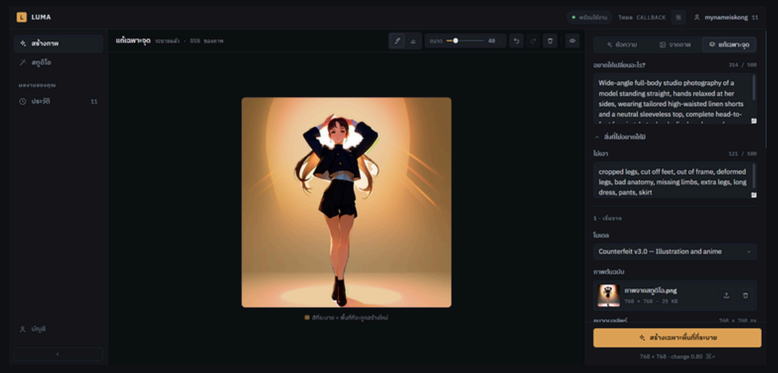
ใช้ 3 เครื่องมือต่อกัน แถบเวอร์ชันมี 4 เวอร์ชัน ลากเส้นแบ่งดูความต่าง



TC-16

ตัดฉาก

พื้นลายตาราง = โปร่งใส ได้ mask ขาวดำมาด้วย (ขาว = ตัวแบบ)

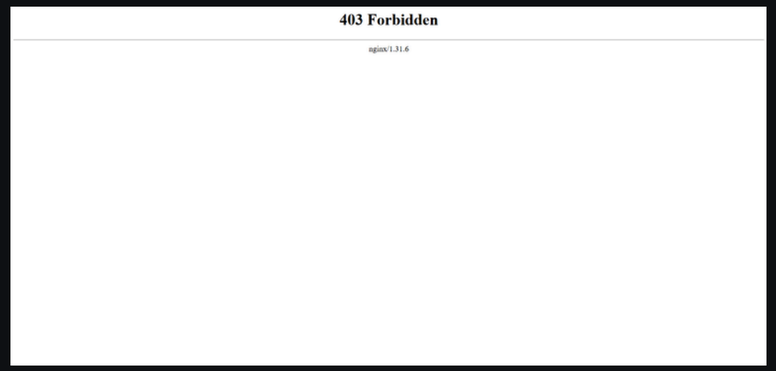


TC-17

วาดฉากหลังใหม่

เปิดโหมดแก้เฉพาะจุดพร้อมภาพต้นฉบับและ mask ฉากหลัง (สีส้ม) ให้เลย

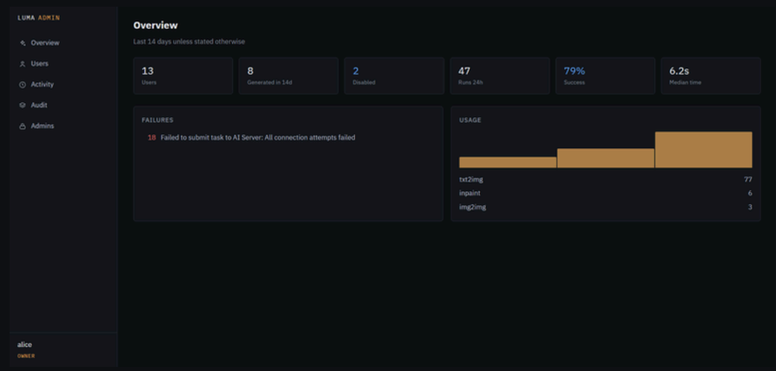
แอดมิน: สิทธิ์เข้า ภาพรวม และข้อมูลผู้ใช้



TC-18 · กรณีผิดพลาด

ผู้ใช้ทั่วไปเปิดแอดมิน

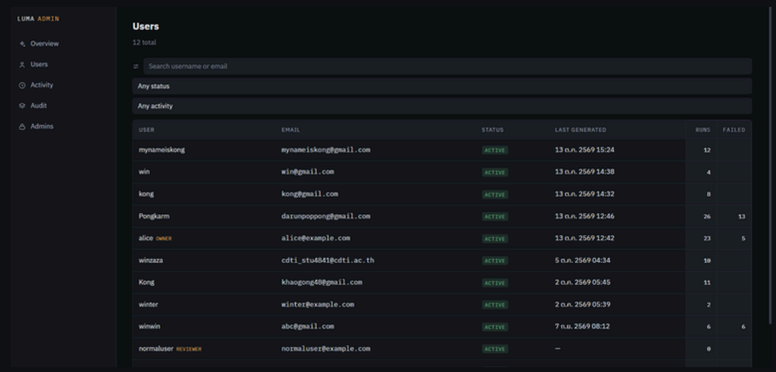
บัญชี demo เห็น "You do not have access" เพราะ backend ตอบ 403 ที่ /admin/me



TC-19

Overview

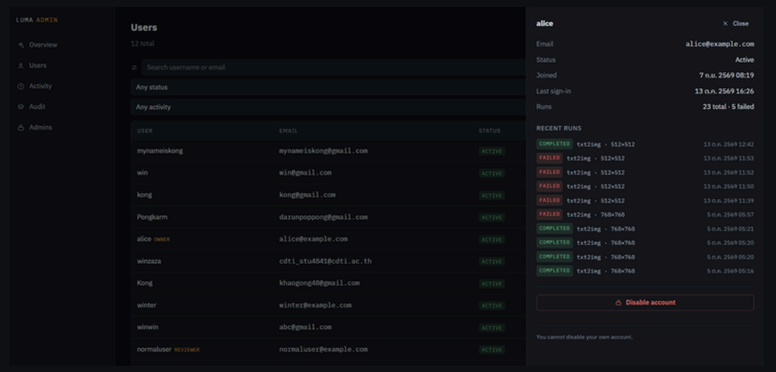
6 ตัวเลขสรุป สาเหตุที่งานล้มเหลว และการใช้งานแต่ละโหมด



TC-20

Users

ค้นหา demo แล้วเปิดแผงรายละเอียด: สถานะ การเข้าใช้ล่าสุด งานล่าสุด

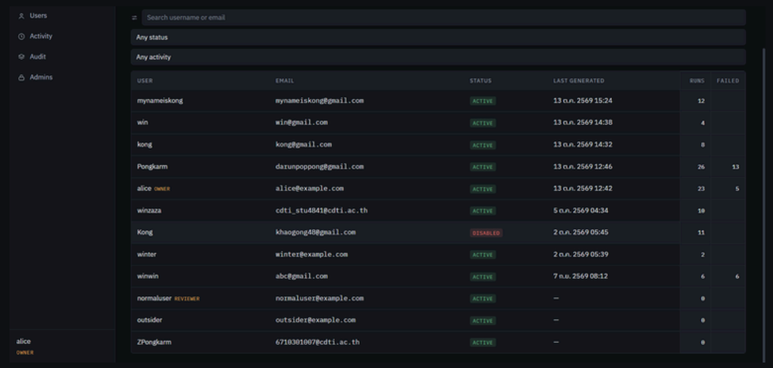


TC-21 · กรณีผิดพลาด

เหตุผลสั้นเกิน

ปิดบัญชีต้องพิมพ์เหตุผลอย่างน้อย 3 ตัวอักษร พิมพ์ ab ยังยืนยันไม่ได้

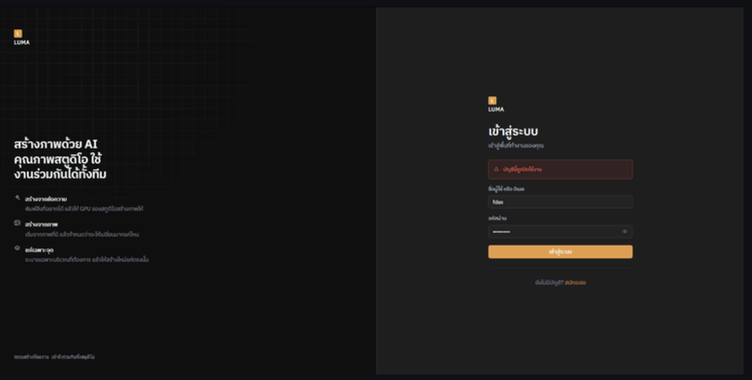
แอดมิน: ปิดบัญชี บันทึกการกระทำ และให้สิทธิ์



TC-22

ปิดบัญชีแล้ว

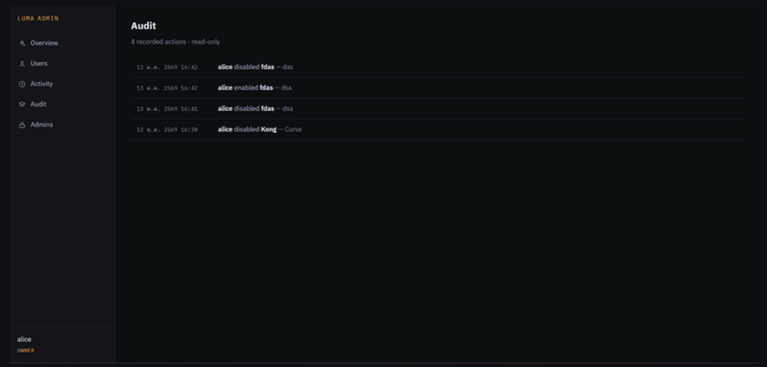
พิมพ์เหตุผลครบแล้วกดยืนยัน สถานะของ demo เปลี่ยนเป็น DISABLED



TC-23 · กรณีผิดพลาด

บัญชีที่ถูกปิด

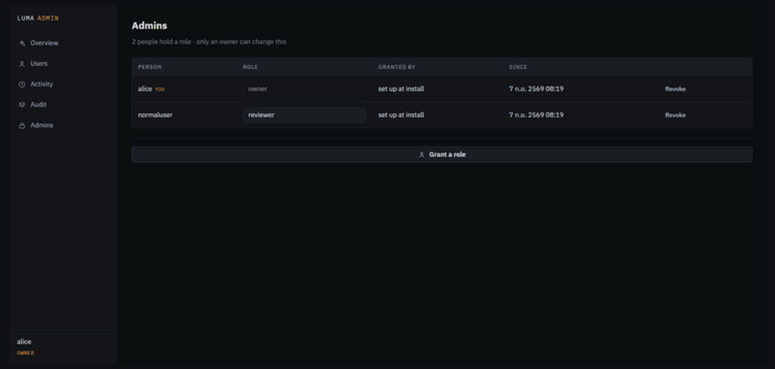
demo ล็อกอินแล้วเห็น "บัญชีนี้ถูกปิดใช้งาน" เปิดคืนแล้วกลับมาใช้ได้



TC-24

Audit

บันทึกทุกการปิด/เปิดบัญชี พร้อมคนทำ เหตุผล และเวลา



TC-25

ให้สิทธิ์ (owner)

tester01 ได้ reviewer ส่วนแถวของ alice ถูกล็อก ลดสิทธิ์ตัวเองไม่ได้

F · ปิดท้าย

ลำดับเดโมสด

01	เปิด http://10.23.130.212 → login	สไลด์ 9
02	txt2img + เปลี่ยนโมเดลให้เห็น LoRA ถูกกรอง	11–12
03	เปิด log ของ Nginx กับคิว AI คู่กัน ให้เห็นงานวิ่งข้ามเครื่อง	4, 21
04	Studio: remove-bg → ส่งไป inpaint ฉากหลัง	13, 19B
05	ประวัติ → "ใช้ค่าเดิม" สร้างซ้ำ	9
06	รัน run_tests.bat ให้เห็น QA 12 ข้อ + dashboard	8

เช็กก่อนขึ้นเวที

- □ 4 เครื่องอยู่วง 10.23.130.x
- □ Backend ตั้ง AI_MODE=callback · secret สองฝั่งตรงกัน
- □ Forge โหลดโมเดลแล้ว (ภาพแรกช้าเพราะต้องโหลด)
- □ เปิด /admin.html ผ่าน Nginx ได้ (แก้แล้ว)
- □ QA ผ่าน 12/12 ก่อนเริ่ม

Q&A

ถาม-ตอบ · ขอบคุณครับ